

TRUE FINAL REWRITE — RUNTIME IMPLEMENTATION

src/core/state.rs

```
use std::collections::HashMap;

#[derive(Clone)]
pub struct AppState {
    pub markets: HashMap<String, MarketSnapshot>,

    pub orders: HashMap<String, DomainOrder>,

    pub trades: HashMap<String, TradeModel>,

    pub market_ordering: Vec<String>,

    pub version: u64,
}

impl Default for AppState {
    fn default() -> Self {
        Self {
            markets: HashMap::new(),

            orders: HashMap::new(),

            trades: HashMap::new(),

            market_ordering: Vec::new(),

            version: 0,
        }
    }
}
```

src/core/shared_state.rs

```
use std::sync::Arc;

use arc_swap::ArcSwap;

use crate::core::state::AppState;

pub type SharedState = Arc<ArcSwap<AppState>>;

pub fn create_shared_state() -> SharedState {
    Arc::new(
        ArcSwap::from_pointee(
            AppState::default()
        )
    )
}
```

src/core/models/domain_order.rs

```
#[derive(Debug, Clone)]
pub struct DomainOrder {
    pub order_id: String,

    pub client_order_id: String,

    pub market_id: String,

    pub side: OrderSide,

    pub price: f64,

    pub quantity: f64,

    pub filled_quantity: f64,

    pub status: OrderStatus,

    pub created_at_ms: u64,

    pub exchange_updated_at_ms: u64,
}
```

```
#[derive(Debug, Clone)]
pub enum OrderSide {
    Buy,
    Sell,
}

#[derive(Debug, Clone)]
pub enum OrderStatus {
    Pending,

    Open,

    PartiallyFilled,

    Filled,

    Cancelled,

    Rejected,
}
```

src/core/models/ui_order_state.rs

```
#[derive(Debug, Clone)]
pub struct UiOrderState {
    pub selected: bool,

    pub editing: bool,

    pub rapid_sell_enabled: bool,

    pub pending_cancel: bool,
}
```

src/core/models/execution_order_state.rs

```
#[derive(Debug, Clone)]
pub struct ExecutionOrderState {
    pub local_pending: bool,
```

```
    pub awaiting_exchange_ack: bool,  
  
    pub optimistic_fill: bool,  
}
```

src/core/events.rs

```
#[derive(Debug, Clone)]  
pub enum AppEvent {  
    Market(MarketEvent),  
  
    Order(OrderEvent),  
  
    Trade(TradeEvent),  
  
    Ui(UiEvent),  
  
    System(SystemEvent),  
}  
  
#[derive(Debug, Clone)]  
pub enum MarketEvent {  
    BidUpdated {  
        market_id: String,  
        bid: f64,  
    },  
  
    AskUpdated {  
        market_id: String,  
        ask: f64,  
    },  
  
    LastTradeUpdated {  
        market_id: String,  
        price: f64,  
    },  
  
    MarketConnected {  
        market_id: String,  
    },  
  
    MarketDisconnected {  
        market_id: String,  
    },  
}
```

```

    },
}

#[derive(Debug, Clone)]
pub enum OrderEvent {
    OrderUpdated {
        order: DomainOrder,
    },

    OrderRemoved {
        order_id: String,
    },

    OrderRejected {
        order_id: String,

        reason: String,
    },
}

#[derive(Debug, Clone)]
pub enum TradeEvent {
    TradeUpdated {
        trade: TradeModel,
    },
}

#[derive(Debug, Clone)]
pub enum UiEvent {
    PlaceOrder {
        market_id: String,

        side: OrderSide,

        price: f64,

        quantity: f64,
    },

    CancelOrder {
        order_id: String,
    },

    SelectMarket {
        market_id: String,
    },

    Refresh,

```

```
}

#[derive(Debug, Clone)]
pub enum SystemEvent {
    Shutdown,

    Error(String),
}
```

src/core/reducer.rs

```
use std::sync::Arc;

use crate::core::events::{
    AppEvent,
    MarketEvent,
    OrderEvent,
    TradeEvent,
};

use crate::core::shared_state::SharedState;

pub fn reduce(
    shared: &SharedState,
    event: AppEvent,
) {
    let current = shared.load();

    let mut next = (*current).clone();

    match event {
        AppEvent::Market(event) => {
            apply_market(
                &mut next,
                event,
            );
        }

        AppEvent::Order(event) => {
            apply_order(
                &mut next,
                event,
            );
        }
    }
}
```

```

AppEvent::Trade(event) => {
    apply_trade(
        &mut next,
        event,
    );
}

AppEvent::Ui(_) => {}

AppEvent::System(_) => {}
}

next.version += 1;

shared.store(
    Arc::new(next)
);
}

fn apply_market(
    state: &mut AppState,
    event: MarketEvent,
) {
    match event {
        MarketEvent::BidUpdated {
            market_id,
            bid,
        } => {
            if let Some(market) =
                state.markets.get_mut(&market_id)
            {
                market.bid = bid;
            }
        }

        MarketEvent::AskUpdated {
            market_id,
            ask,
        } => {
            if let Some(market) =
                state.markets.get_mut(&market_id)
            {
                market.ask = ask;
            }
        }

        MarketEvent::LastTradeUpdated {

```

```

        market_id,
        price,
    } => {
        if let Some(market) =
            state.markets.get_mut(&market_id)
        {
            market.last_price = price;
        }
    }

    _ => {}
}

fn apply_order(
    state: &mut AppState,
    event: OrderEvent,
) {
    match event {
        OrderEvent::OrderUpdated {
            order,
        } => {
            state.orders.insert(
                order.order_id.clone(),
                order,
            );
        }

        OrderEvent::OrderRemoved {
            order_id,
        } => {
            state.orders.remove(
                &order_id
            );
        }

        _ => {}
    }
}

fn apply_trade(
    state: &mut AppState,
    event: TradeEvent,
) {
    match event {
        TradeEvent::TradeUpdated {
            trade,
        } => {

```

```

        state.trades.insert(
            trade.trade_id.clone(),
            trade,
        );
    }
}
}

```

src/worker/reducer_task.rs

```

use tokio::sync::mpsc::Receiver;

use crate::core::events::AppEvent;

use crate::core::reducer::reduce;

use crate::core::shared_state::SharedState;

const MAX_BATCH: usize = 256;

pub async fn start_reducer_task(
    shared: SharedState,

    mut rx: Receiver<AppEvent>,
) {
    let mut batch =
        Vec::with_capacity(MAX_BATCH);

    loop {
        batch.clear();

        let first = match rx.recv().await {
            Some(event) => event,

            None => break,
        };

        batch.push(first);

        while let Ok(event) =
            rx.try_recv()
        {
            batch.push(event);
        }
    }
}

```

```

        if batch.len() >= MAX_BATCH {
            break;
        }
    }

    for event in batch.drain(..) {
        reduce(
            &shared,
            event,
        );
    }
}
}

```

src/infrastructure/repaint_scheduler.rs

```

use std::sync::atomic::{
    AtomicBool,
    AtomicU64,
    Ordering,
};

use std::time::{
    Duration,
    SystemTime,
    UNIX_EPOCH,
};

pub struct RepaintScheduler {
    dirty: AtomicBool,

    queued: AtomicBool,

    last_frame_ms: AtomicU64,

    min_frame_interval_ms: u64,
}

impl RepaintScheduler {
    pub fn new(
        min_frame_interval_ms: u64,
    ) -> Self {
        Self {
            dirty: AtomicBool::new(false),

```

```

        queued: AtomicBool::new(false),

        last_frame_ms:
            AtomicU64::new(0),

        min_frame_interval_ms,
    }
}

pub fn mark_dirty(&self) {
    self.dirty.store(
        true,
        Ordering::Relaxed,
    );
}

pub fn should_repaint(
    &self,
) -> bool {
    if !self
        .dirty
        .load(Ordering::Relaxed)
    {
        return false;
    }

    let now = current_time_ms();

    let last = self
        .last_frame_ms
        .load(Ordering::Relaxed);

    if now.saturating_sub(last)
        < self.min_frame_interval_ms
    {
        return false;
    }

    self.last_frame_ms.store(
        now,
        Ordering::Relaxed,
    );

    self.dirty.store(
        false,
        Ordering::Relaxed,
    );
}

```

```

        true
    }
}

fn current_time_ms() -> u64 {
    SystemTime::now()
        .duration_since(
            UNIX_EPOCH
        )
        .unwrap_or(Duration::ZERO)
        .as_millis() as u64
}

```

src/app/snapshot/snapshot_cache.rs

```

pub struct SnapshotCache {
    last_version: u64,

    snapshot: UiSnapshot,
}

impl SnapshotCache {
    pub fn new() -> Self {
        Self {
            last_version: 0,

            snapshot: UiSnapshot {
                markets: Vec::new(),

                orders: Vec::new(),

                trades: Vec::new(),
            },
        }
    }

    pub fn snapshot(
        &mut self,

        state: &AppState,
    ) -> &UiSnapshot {
        if state.version
            != self.last_version

```

```

    {
        self.snapshot =
            UiSnapshot {
                markets: state
                    .markets
                    .values()
                    .cloned()
                    .collect(),

                orders: state
                    .orders
                    .values()
                    .cloned()
                    .collect(),

                trades: state
                    .trades
                    .values()
                    .cloned()
                    .collect(),
            };

        self.last_version =
            state.version;
    }

    &self.snapshot
}

```

src/worker/priority_queues.rs

```

use tokio::sync::mpsc;

use crate::core::events::AppEvent;

pub struct PriorityQueues {
    pub high_priority_tx:
        mpsc::Sender<AppEvent>,

    pub high_priority_rx:
        mpsc::Receiver<AppEvent>,

    pub low_priority_tx:

```

```

        mpsc::Sender<AppEvent>,

    pub low_priority_rx:
        mpsc::Receiver<AppEvent>,
}

impl PriorityQueues {
    pub fn new() -> Self {
        let (
            high_priority_tx,
            high_priority_rx,
        ) = mpsc::channel(4096);

        let (
            low_priority_tx,
            low_priority_rx,
        ) = mpsc::channel(16384);

        Self {
            high_priority_tx,

            high_priority_rx,

            low_priority_tx,

            low_priority_rx,
        }
    }
}

```